

Q.1) Add # Symbol at the end of payment .

SELECT *, CONCAT((payment_status),"#") AS PAYMENT FROM dmart_orders;

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL query:

```
SELECT *, CONCAT((payment_status),"#") AS PAYMENT FROM dmart_orders;
```

The 'Result Grid' shows the output of the query, which includes columns for order_id, customer_id, customer_name, product_category, quantity, price_per_unit, order_date, payment_status, and PAYMENT. The PAYMENT column contains the payment status followed by a hash symbol (#).

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status	PAYMENT
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid	Paid#
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid	Paid#
1003	C103	AMIT MISHRA	Apparel	3	150.20	2026-01-02	Pending	Pending#
1004	C101	Rohan Sharma	Packaged Foods	10	22.45	2026-01-02	Paid	Paid#
1005	C104	Sneha Reddy	Personal Care	2	85.33	2026-01-03	Paid	Paid#
1006	C105	Vikram Singh	Groceries	12	10.00	2026-01-03	Refunded	Refunded#

The 'Output' tab shows the execution log, indicating that the query was executed successfully and returned 100 rows.

Insights:

This query adds # symbol at the end of payment status.

Q.2) Print initial of product category.

SELECT *, SUBSTR(product_category,1,1) AS INITIAL FROM dmart_orders;

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying a SQL query that selects all columns from the 'dmart_orders' table and adds a new column 'INITIAL' which is the first character of the 'product_category' column. The query is as follows:

```
SELECT *, SUBSTR(product_category,1,1) AS INITIAL FROM dmart_orders;
```

The 'Result Grid' shows the results of the query. The columns are: order_id, customer_id, customer_name, product_category, quantity, price_per_unit, order_date, payment_status, and INITIAL. The results are as follows:

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status	INITIAL
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid	G
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid	E
1003	C103	AMIT MISHRA	Apparel	3	150.20	2026-01-02	Pending	A
1004	C101	Rohan Sharma	Packaged Foods	10	22.45	2026-01-02	Paid	P
1005	C104	Sneha Reddy	Personal Care	2	85.33	2026-01-03	Paid	P
1006	C105	Vikram Singh	Groceries	12	10.00	2026-01-03	Refunded	G

The 'Output' tab shows the execution of the query. The actions are as follows:

#	Time	Action	Message	Duration / Fetch
14	20:52:14	SELECT *, CONCAT(payment_status,'#') AS PAYMENT FROM dmart_orders LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec
15	20:52:43	SELECT *, CONCAT(payment_status,'#') AS PAYMENT FROM dmart_orders LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec
16	20:53:09	SELECT *, UPPER(customer_name) AS UPPER FROM dmart_orders LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec
17	20:53:34	SELECT *, LOWER(customer_name) AS LOWER FROM dmart_orders LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec
18	20:55:21	SELECT *, SUBSTR(customer_name,1,1) AS INITIAL FROM dmart_orders LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec
19	20:55:59	SELECT *, SUBSTR(product_category,1,1) AS INITIAL FROM dmart_orders LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec

Insights:

It is useful for sorting the product category.

Q.3) List orders where price per unit is Between 50 and 100.

SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit BETWEEN 50 AND 100;

MySQL Workbench

unconnected x MySQL@127.0.0.1:3306 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 x

MANAGEMENT

- Server Status
- Client Connections
- Users and Privilege
- Status and System
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Replic
- Performance Scher

SQL: dmart_orders

```

110 (1096, 'C115', 'Gaurav Kapur', 'Packaged Foods', 9, 19.00, '2026-02-17', 'Paid'),
111 (1097, 'C102', 'Priya Patel', 'Home Appliances', 1, 750.00, '2026-02-16', 'Paid'),
112 (1098, 'C105', 'Vikram Singh', 'Apparel', 3, 160.00, '2026-02-18', 'Paid'),
113 (1099, 'C108', 'Kiran Joshi', 'Personal Care', 5, 55.40, '2026-02-19', 'Pending'),
114 (1100, 'C116', 'Arjun Kapoor', 'Groceries', 16, 12.50, '2026-02-19', 'Paid');
115 SELECT * FROM dmart_orders;
116 SELECT *, CONCAT((payment_status),'#') AS PAYMENT FROM dmart_orders;
117 SELECT *, UPPER(customer_name) AS UPPER FROM dmart_orders;
118 SELECT *, LOWER(customer_name) AS LOWER FROM dmart_orders;
119 SELECT *, SUBSTR(product_category,1,1) AS INITIAL FROM dmart_orders;
120 SELECT *, SUBSTR(product_category,2) AS NO_INITIAL FROM dmart_orders;
121 SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 50;
122 SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit BETWEEN 50 AND 100;

```

Result Grid

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status	GREATER_THAN_50
1005	C104	Sneha Reddy	Personal Care	2	85.33	2026-01-03	Paid	85.33
1008	C106	Anjali Desai	Apparel	4	99.99	2026-01-04	Pending	99.99
1014	C101	Rohan Sharma	Groceries	2	65.40	2026-01-07	Paid	65.40
1022	C108	Kiran Joshi	Groceries	4	55.00	2026-01-11	Paid	55.00
1026	C114	Ritu Phogat	Personal Care	2	95.50	2026-01-13	Paid	95.50
1034	C113	Deepak Gupta	Personal Care	4	60.00	2026-01-17	Paid	60.00

Output

#	Time	Action	Message	Duration / Fetch
19	20:55:59	SELECT *, SUBSTR(product_category,1,1) AS INITIAL FROM dmart_orders LIMIT 0.1000	100 row(s) returned	0.000 sec / 0.000 sec
20	20:57:27	SELECT *, SUBSTR(product_category,2) AS NO_INITIAL FROM dmart_orders LIMIT 0.1000	100 row(s) returned	0.000 sec / 0.000 sec
21	20:59:08	SELECT *, SUBSTR(product_category,2) AS NO_INITIAL FROM dmart_orders LIMIT 0.1000	100 row(s) returned	0.016 sec / 0.000 sec
22	21:03:22	SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 50 LIMIT 0.1000	54 row(s) returned	0.000 sec / 0.000 sec
23	21:03:45	SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 50 LIMIT 0.1000	54 row(s) returned	0.016 sec / 0.000 sec
24	21:05:06	SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit BETWEEN 50 AND 100 LIMIT 0.1000	15 row(s) returned	0.000 sec / 0.000 sec

Query Completed

Insights:

It is useful for find the price between selected amount.

Q.4) Find price per unit >10 and product="Groceries".

```
SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE  
price_per_unit > 10 AND product_category='Groceries';
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to select all columns from the `dmart_orders` table, with the `price_per_unit` column aliased as `GREATER_THAN_50`. The query filters for `price_per_unit > 10` and `product_category = 'Groceries'`. The results are displayed in a table with columns: `order_id`, `customer_id`, `customer_name`, `product_category`, `quantity`, `price_per_unit`, `order_date`, `payment_status`, and `GREATER_THAN_50`. The results show 23 rows of data for Groceries orders. The bottom panel shows the execution log with the query being executed at 21:07:48, returning 20 rows.

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status	GREATER_THAN_50
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid	45.50
1010	C107	Raj Mahotra	Groceries	8	35.25	2026-01-05	Paid	35.25
1014	C101	Rohan Sharma	Groceries	2	65.40	2026-01-07	Paid	65.40
1022	C108	kiran joshi	Groceries	4	55.00	2026-01-11	Paid	55.00
1025	C103	AMIT MISHRA	Groceries	7	42.10	2026-01-13	Paid	42.10
1030	C106	Anjali Desai	Groceries	11	14.50	2026-01-15	Pending	14.50

Insights:

Displays the product price greater than Rs.10.

Q.5) List all categories Without Repetation.

SELECT DISTINCT product_category FROM dmart_orders;

The screenshot shows the MySQL Workbench interface. The 'Query Editor' tab is active, displaying a SQL query that lists various database operations, including a final query: `SELECT DISTINCT product_category FROM dmart_orders;`. The 'Result Grid' tab is also visible, showing the output of the query. The output is a table with one column, 'product_category', and several rows of data. The data includes categories like 'Groceries', 'Electronics', 'Apparel', 'Packaged Foods', 'Personal Care', and 'Home Appliances'. The 'Output' tab is also visible, showing a log of actions and their results.

product_category
Groceries
Electronics
Apparel
Packaged Foods
Personal Care
Home Appliances

Insights:

It gives the all product categories present in shop.

Q.6) Find product category Electronics and Groceries.

```
SELECT * FROM dmart_orders WHERE UPPER(product_category) IN ("ELECTRONICS","GROCERIES");
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to select all orders where the product category is either Electronics or Groceries. The query is as follows:

```
118 SELECT *, LOWER(customer_name) AS LOWER FROM dmart_orders;
119 SELECT *, SUBSTR(product_category,1,1) AS INITIAL FROM dmart_orders;
120 SELECT *, SUBSTR(product_category,2) AS NO_INITIAL FROM dmart_orders;
121 SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 50;
122 SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit BETWEEN 50 AND 100;
123 SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 10 AND product_category='Groceries';
124 # SELECT * FROM dmart_orders WHERE customer_name IN ('Pooja','Kiran');
125 SELECT * FROM dmart_orders WHERE UPPER(customer_name)= "AMIT";
126 SELECT * FROM dmart_orders WHERE UPPER(customer_name) LIKE "AS";
127 SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "PS";
128 SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "Kson%";
129 SELECT DISTINCT product_category FROM dmart_orders;
130 SELECT * FROM dmart_orders WHERE UPPER(product_category) IN ("ELECTRONICS","GROCERIES");
```

The Results grid shows the output of the query, displaying columns: order_id, customer_id, customer_name, product_category, quantity, price_per_unit, order_date, and payment_status. The results are as follows:

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid
1006	C105	Vikram Singh	Groceries	12	10.00	2026-01-03	Refunded
1009	C103	AMIT MISHRA	Electronics	2	300.50	2026-01-05	Paid
1010	C107	Raj Malhotra	Groceries	8	35.25	2026-01-05	Paid
1014	C101	Rohan Sharma	Groceries	2	65.40	2026-01-07	Paid

The Output pane shows the execution log, including the query execution time and the number of rows returned. The final query (SELECT * FROM dmart_orders WHERE UPPER(product_category) IN ("ELECTRONICS","GROCERIES")) returned 36 rows.

Insights:

It saves time to find specific 2 or more categories.

Q.7) Find product category without including Electronics and Groceries.

```
SELECT * FROM dmart_orders WHERE UPPER(product_category) NOT IN ("ELECTRONICS","GROCERIES");
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to select all orders where the product category is not 'ELECTRONICS' or 'GROCERIES'. The results grid shows a list of orders with columns: order_id, customer_id, customer_name, product_category, quantity, price_per_unit, order_date, and payment_status. The output pane shows the execution of the query, indicating that 64 rows were returned.

```
119 SELECT *, SUBSTR(product_category,1,1) AS INITIAL FROM dmart_orders;
120 SELECT *, SUBSTR(product_category,2) AS NO_INITIAL FROM dmart_orders;
121 SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 50;
122 SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit BETWEEN 50 AND 100;
123 SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 10 AND product_category='groceries';
124 # SELECT * FROM dmart_orders WHERE customer_name IN ('Pooja','Kiran');
125 SELECT * FROM dmart_orders WHERE UPPER(customer_name)= "AMIT";
126 SELECT * FROM dmart_orders WHERE UPPER(customer_name) LIKE "A%";
127 SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "P%";
128 SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "kson%";
129 SELECT DISTINCT product_category FROM dmart_orders;
130 SELECT * FROM dmart_orders WHERE UPPER(product_category) IN ("ELECTRONICS","GROCERIES");
131 SELECT * FROM dmart_orders WHERE UPPER(product_category) NOT IN ("ELECTRONICS","GROCERIES");
```

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status
1003	C103	AMIT MISHRA	Apparel	3	150.20	2026-01-02	Pending
1004	C101	Rohan Sharma	Packaged Foods	10	22.45	2026-01-02	Paid
1005	C104	Sneha Reddy	Personal Care	2	85.33	2026-01-03	Paid
1007	C102	Priya Patel	Home Appliances	1	450.00	2026-01-04	Paid
1008	C106	Anjali Desai	Apparel	4	99.99	2026-01-04	Pending
1011	C108	Kiran Joshi	Dairy	6	15.80	2026-01-06	Paid

dmart_orders 44 %

Output

#	Time	Action	Message	Duration / Fetch
7	21:36:18	SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "P%"; LIMIT 0, 1000	29 row(s) returned	0.000 sec / 0.000 sec
8	21:37:34	SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "kson%"; LIMIT 0, 1000	13 row(s) returned	0.000 sec / 0.000 sec
9	21:38:34	SELECT DISTINCT product_category FROM dmart_orders LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec
10	21:40:07	SELECT * FROM product_category WHERE UPPER(product_category) IN ("ELECTRONICS","GROCERIES") LIMIT...	Error Code: 1146. Table 'd_mart2.product_category' doesn't exist	0.000 sec
11	21:40:21	SELECT * FROM dmart_orders WHERE UPPER(product_category) IN ("ELECTRONICS","GROCERIES") LIMIT 0, 1...	36 row(s) returned	0.000 sec / 0.000 sec
12	21:41:09	SELECT * FROM dmart_orders WHERE UPPER(product_category) NOT IN ("ELECTRONICS","GROCERIES") LIM...	64 row(s) returned	0.016 sec / 0.000 sec

Query Completed

Insights:

It is useful to find other than this categories.

Q.8) Show payment_status pending orders.

SELECT * FROM dmart_orders WHERE UPPER(payment_status) = "PENDING";

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to select all orders with a pending payment status. The query is as follows:

```
120. SELECT *, SUBSTR(product_category,2) AS NO_INITIAL FROM dmart_orders;
121. SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 50;
122. SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit BETWEEN 50 AND 100;
123. SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 10 AND product_category='Groceries';
124. # SELECT * FROM dmart_orders WHERE customer_name IN ('Pooja','Kiran');
125. SELECT * FROM dmart_orders WHERE UPPER(customer_name)= "AMIT";
126. SELECT * FROM dmart_orders WHERE UPPER(customer_name) LIKE "AS";
127. SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "PS";
128. SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "Kson";
129. SELECT DISTINCT product_category FROM dmart_orders;
130. SELECT * FROM dmart_orders WHERE UPPER(product_category) IN ("ELECTRONICS","GROCERIES");
131. SELECT * FROM dmart_orders WHERE UPPER(product_category) NOT IN ("ELECTRONICS","GROCERIES");
132. SELECT * FROM dmart_orders WHERE UPPER(payment_status) = "PENDING";
```

The Results grid shows the following data:

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status
1003	C103	AMIT MISHRA	Apparel	3	150.20	2026-01-02	Pending
1008	C106	Anjali Desai	Apparel	4	99.99	2026-01-04	Pending
1013	C109	Nitesh Verma	Personal Care	1	120.00	2026-01-07	Pending
1020	C107	Raj Malhotra	Apparel	1	250.00	2026-01-10	Pending
1030	C106	Anjali Desai	Groceries	11	14.50	2026-01-15	Pending
1039	C118	Varun Dhawan	Packaged Foods	12	15.00	2026-01-20	Pending
1046	C109	Nitesh Verma	Apparel	6	85.00	2026-01-22	Pending

The Output tab shows the execution of the query, indicating that 13 rows were returned.

Insights:

It is useful for finding Paid or pending orders.

Q.9) Remove the blank spaces from customer name.

```
SELECT * FROM dmart_orders WHERE TRIM(UPPER(customer_name)) = "ROHAN SHARMA";
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to select all records from the `dmart_orders` table where the customer name, after trimming and converting to uppercase, matches "ROHAN SHARMA". The query is as follows:

```
121. SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 50;
122. SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit BETWEEN 50 AND 100;
123. SELECT *, price_per_unit AS GREATER_THAN_50 FROM dmart_orders WHERE price_per_unit > 10 AND product_category='Groceries';
124. # SELECT * FROM dmart_orders WHERE customer_name IN ('Pooja','Kiran');
125. SELECT * FROM dmart_orders WHERE UPPER(customer_name) = "AMIT";
126. SELECT * FROM dmart_orders WHERE UPPER(customer_name) LIKE "AS";
127. SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "PA";
128. SELECT * FROM dmart_orders WHERE UPPER(product_category) LIKE "sonN";
129. SELECT DISTINCT product_category FROM dmart_orders;
130. SELECT * FROM dmart_orders WHERE UPPER(product_category) IN ("ELECTRONICS","GROCERIES");
131. SELECT * FROM dmart_orders WHERE UPPER(product_category) NOT IN ("ELECTRONICS","GROCERIES");
132. SELECT * FROM dmart_orders WHERE UPPER(payment_status) = "PENDING";
133. SELECT * FROM dmart_orders WHERE TRIM(UPPER(customer_name)) = "ROHAN SHARMA";
```

The Results grid shows the output of the query, displaying columns: `order_id`, `customer_id`, `customer_name`, `product_category`, `quantity`, `price_per_unit`, `order_date`, and `payment_status`. The results show 7 rows for the customer "ROHAN SHARMA".

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid
1004	C101	Rohan Sharma	Packaged Foods	10	22.45	2026-01-02	Paid
1014	C101	Rohan Sharma	Groceries	2	65.40	2026-01-07	Paid
1028	C101	Rohan Sharma	Home Appliances	1	520.00	2026-01-26	Paid
1051	C101	Rohan Sharma	Electronics	1	1350.00	2026-01-26	Paid
1071	C101	Rohan Sharma	Dairy	6	21.50	2026-02-05	Paid
1081	C101	Rohan Sharma	Packaged Foods	5	20.00	2026-01-15	Paid

The Action Output pane shows the execution of the query, indicating that 7 rows were returned.

#	Time	Action	Message	Duration / Fetch
13	21:42:10	SELECT * FROM dmart_orders WHERE UPPER(product_category) = "ELECTRONICS" LIMIT 0, 1000	0 row(s) returned	0.016 sec / 0.000 sec
14	21:42:30	SELECT * FROM dmart_orders LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec
15	21:42:58	SELECT * FROM dmart_orders WHERE UPPER(payment_status) = "PAID" LIMIT 0, 1000	77 row(s) returned	0.016 sec / 0.000 sec
16	21:43:09	SELECT * FROM dmart_orders WHERE UPPER(payment_status) = "PENDING" LIMIT 0, 1000	13 row(s) returned	0.000 sec / 0.000 sec
17	21:44:14	SELECT * FROM dmart_orders LIMIT 0, 1000	100 row(s) returned	0.016 sec / 0.000 sec
18	21:44:52	SELECT * FROM dmart_orders WHERE TRIM(UPPER(customer_name)) = "ROHAN SHARMA" LIMIT 0, 1000	7 row(s) returned	0.016 sec / 0.000 sec

Insights:

It removes blank spaces from name for easy access of name without error.

Q.10) Add column Shopping in table where quantity <2 then print Small , quantity>2 and quantity <5 print Medium else large using case statement.

```
SELECT *,CASE WHEN quantity < 2 THEN "SMALL"
```

```
WHEN quantity>2 AND quantity<5 THEN "MEDIUM"
```

```
ELSE "LARGE"
```

```
END AS SHOPPING
```

```
FROM dmart_orders;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that adds a 'SHOPPING' column to the 'dmart_orders' table based on the quantity of items. The query is as follows:

```
SELECT *,CASE WHEN quantity < 2 THEN "SMALL"
WHEN quantity>2 AND quantity<5 THEN "MEDIUM"
ELSE "LARGE"
END AS SHOPPING
FROM dmart_orders;
```

The results grid shows the following data:

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status	SHOPPING
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid	LARGE
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid	SMALL
1003	C103	AMIT MISHRA	Apparel	3	150.20	2026-01-02	Pending	MEDIUM
1004	C101	Rohan Sharma	Packaged Foods	10	22.45	2026-01-02	Paid	LARGE
1005	C104	Shreya Reddy	Personal Care	2	85.33	2026-01-03	Paid	LARGE
1006	C105	Vikram Singh	Groceries	12	10.00	2026-01-03	Refunded	LARGE

The output pane shows the execution of the query, with messages indicating that 100 rows were returned for each of the six queries executed.

Insights:

It is useful for sorting the data according to shopping.

Q.11) Add column shopping in table , if quantity <2 print small, quantity>2 and less than 5 print Medium and else print Large.

```
SELECT *,
```

```
IF (quantity < 2,"SMALL",
```

```
IF (quantity > 2 AND quantity<5 , "MIDIUM",
```

```
"LARGE")) AS SHOPPING
```

```
FROM dmart_orders;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
136 ELSE "LARGE"
137 END AS SHOPPING
138 FROM dmart_orders ;
139 SELECT *,CASE WHEN quantity < 2 THEN "SMALL"
140 WHEN quantity>2 AND quantity<5 THEN "MIDIUM"
141 ELSE "LARGE"
142 END AS SHOPPING
143 FROM dmart_orders;
144 SELECT *,
145 IF (quantity < 2,"SMALL",
146 IF (quantity > 2 AND quantity<5 , "MIDIUM",
147 "LARGE")) AS SHOPPING
148 FROM dmart_orders;
```

The Results tab shows the output of the query, which includes a new column 'SHOPPING' with values 'SMALL', 'MIDIUM', or 'LARGE' based on the quantity of items in each order.

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status	SHOPPING
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid	LARGE
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid	SMALL
1003	C103	AMIT MISHRA	Apparel	3	150.20	2026-01-02	Pending	MIDIUM
1004	C101	Rohan Sharma	Packaged Foods	10	22.45	2026-01-02	Paid	LARGE
1005	C104	Shruti Reddy	Personal Care	2	85.33	2026-01-03	Paid	LARGE
1006	C105	Vikram Singh	Groceries	12	10.00	2026-01-03	Refunded	LARGE

The Output tab shows the execution log, indicating that the query was executed successfully and returned 100 rows.

Insights:

It is useful for sorting the data according to shopping.

Q.12) Find the Repeated customers.

SELECT* FROM dmart_orders WHERE price_per_unit >300 ORDER BY customer_id;

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
137 END AS SHOPPING
138 FROM dmart_orders ;
139 SELECT *,CASE WHEN quantity < 2 THEN "SMALL"
140 WHEN quantity>2 AND quantity<5 THEN "MIDIUM"
141 ELSE "LARGE"
142 END AS SHOPPING
143 FROM dmart_orders;
144 SELECT *,
145 IF (quantity < 2,"SMALL",
146 IF (quantity > 2 AND quantity<5 , "MIDIUM",
147 "LARGE")) AS SHOPPING
148 FROM dmart_orders;
149 SELECT* FROM dmart_orders WHERE price_per_unit >300 ORDER BY customer_id;
```

The Results grid shows the following data:

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status
1028	C101	Rohan Sharma	Home Appliances	1	520.00	2026-01-14	Paid
1051	C101	Rohan Sharma	Electronics	1	1350.00	2026-01-26	Paid
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid
1007	C102	Priya Patel	Home Appliances	1	450.00	2026-01-04	Paid
1097	C102	Priya Patel	Home Appliances	1	750.00	2026-02-18	Paid
1009	C103	AMIT MISHRA	Electronics	2	300.50	2026-01-05	Paid

The Output pane shows the execution progress of the queries, with the final query (33) returning 21 rows.

Insights:

For finding regular customers of shop.

Q.13) Find the total amount.

SELECT *, (price_per_unit)*quantity AS TOTAL_AMOUNT FROM dmart_orders;

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that categorizes orders by quantity and calculates the total amount for each order. The query is as follows:

```
140 WHEN quantity > 2 AND quantity < 5 THEN "MEDIUM"
141 ELSE "LARGE"
142 END AS SHOPPING
143 FROM dmart_orders;
144 SELECT *,
145 IF (quantity < 2, "SMALL",
146 IF (quantity > 2 AND quantity < 5, "MEDIUM",
147 "LARGE")) AS SHOPPING
148 FROM dmart_orders;
149 SELECT * FROM dmart_orders WHERE price_per_unit > 300 ORDER BY customer_id;
150 SELECT * FROM dmart_orders WHERE UPPER(payment_status) = "PAID";
151 SELECT * FROM dmart_orders WHERE TRIM(UPPER(customer_name)) LIKE "ROHAN SHARMA" AND product_category IN ("GROCERIES");
152 SELECT *, (price_per_unit)*quantity AS TOTAL_AMOUNT FROM dmart_orders;
```

The Results grid shows the following data:

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status	TOTAL_AMOUNT
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid	227.50
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid	1200.75
1003	C103	AMIT MISHRA	Apparel	3	150.20	2026-01-02	Pending	450.60
1004	C101	Rohan Sharma	Packaged Foods	10	22.45	2026-01-02	Paid	224.50
1005	C104	Sneha Reddy	Personal Care	2	85.33	2026-01-03	Paid	170.66
1006	C105	Vikram Singh	Groceries	12	10.00	2026-01-03	Refunded	120.00
1007	C102	Priya Patel	Electronics	1	450.00	2026-01-04	Paid	450.00

The Output tab shows the execution of the query, indicating that 100 rows were returned.

Insights:

It is useful for calculating the revenue of business.